1

1

/ D /

`Login.test.tsx` 132

`test_backend.py` 3

`npm run test:run` `python manage.py test`

-
- A A A
-
- **`Login.test.tsx`** `vi.mock`
  `describe` / `it` / `beforeEach` `render` / `screen` / `fireEvent` / `waitFor` / `findBy`
  `getByRole` / `getByLabelText` `mockResolvedValueOnce` / `mockRejectedValueOnce`
  401
- **`getBy`** **`findBy`** **`queryBy`**
- D `TestCase` `APIClient`
- **`test_backend.py`** **`setUp`** `make_break_times` `BaseTestCase` `login`
- **`TestModels`** `assertEqual` / `assertIn` / `assertTrue`
-
- `npm run test:run` / `python manage.py test --settings=hozen_another.test_settings`
-

0.
1. 3
2.
3.
4. . . 132
5. B / B / B
6.
7. D C A IC
8. .
9.
10. A I 401 / 403 / 200
11.
12. .
13.
14.
15.
16.
17.
18.

>

12345

> 1

	1		
1			

> 1

			`frontend/src/tests/`	`npm run test:run`
	D A I	D `TestCase` `APIClient`	`kosu/tests/`	`python manage.py test ...`

3

```
1.  Arrange /        …          え
                                せ      す      く
2.  Act /            …    し い   1  け う
                              す        API   く
3.  Assert /         …   が    お か   か
                       「～   ず」       し、  え
```

> `expect(x).toBe(5)` `self.assertEqual(x, 5)`

>

```
【  】          く
【  】 「2」「 」「3」「 」   す
【  】     「5」が   さ   い   ず →  え
```

3 1 `Login.test.tsx` `test_backend.py`

2

> .
>
> /
>
> / `describe`

`it` 1 `expect`

`render` `screen` `fireEvent`

D

6

`Login.test.tsx`

`Login.tsx` 3

1. `api.post(...)`
2. `useNavigate()`

3. `import logo from "../../img/TitleRogo.png"`

A I		/
		`logo.png`

>

`vi.fn()`	
`vi.mock("  ", () => ...)`	
`xxx.mockResolvedValueOnce( )`	1 A I
`xxx.mockRejectedValueOnce( )`	1 A I
`vi.clearAllMocks()`	

`frontend/src/tests/MainPage/Login.test.tsx`

401 4

> `Login`

```
// ① Vitest
import { vi, describe, it, expect, beforeEach } from "vitest";
// ② RTL
import { render, screen, fireEvent, waitFor } from "@testing-library/react";
// ③
import { MemoryRouter } from "react-router-dom";
// ④ 「 ○○があ 」 す
import "@testing-library/jest-dom/vitest";
// ⑤
import Login from "../../MainPage/Login";
```

1

- `import { vi, describe, it, expect, beforeEach } from "vitest";` 5
  - `vi` 3
  - `describe`
  - `it`
  - `expect`
  - `beforeEach`
- `import { render, screen, fireEvent, waitFor } from "@testing-library/react";` 4
  - `render`
  - `screen`
  - `fireEvent`
  - `waitFor`
- `import { MemoryRouter } from "react-router-dom";` `MemoryRouter`

Login

Login.tsx <Link to="/help"> useNavigate()

MemoryRouter

- import "@testing-library/jest-dom/vitest"; -

toBeInTheDocument() :oHaveTextContent()

/vitest

- import LoginPage from "../../MainPage/Login";

../../ 2 src/tests/MainPage/

src/MainPage/ 2 MainPage/Login

'

vi.mock("../../

-

> A I api/axios
> / / / 4
> mockedApi

```
// api/axios ご し え
vi.mock("../../api/axios", () => ({
  //  default import api from け
  default: {
    //  get せ
    get: vi.fn(),
    //  post せ う
    post: vi.fn(),
    //  put せ
    put: vi.fn(),
    //  delete せ
    delete: vi.fn(),
  },
}));
//  し え せ api
import api from "../../api/axios";
//  any し mockResolvedValueOnce う
const mockedApi = api as any;
```

1

- **`vi.mock("../../api/axios", () => ({ default: { ... } }))`**
  `api/axios` `07`
  `default` 4
- **`get: vi.fn()`** **`post: vi.fn()`** **`put: vi.fn()`** **`delete: vi.fn()`** `vi.fn()`
  `post` 3 `Login.tsx`
- **`import api from "../../api/axios";`** `vi.mock` `api`
  `vi.mock` `api/axios`
  `api`
- **`const mockedApi = api as any;`** `api` `mockedApi` `as any`

> **as any** mockResolvedValueOnce
> api
> as any

> isAxiosError
> Login.tsx

```
// axios        し  え
vi.mock("axios", () => ({
  default: {
    // 「err    response があ             」       す   せ
    isAxiosError: vi.fn((err) => !!err.response),
  },
}));
```

- **isAxiosError: vi.fn((err) => !!err.response)** isAxiosError
  true/false
  ** err.response true **
  - err.response
  - !! 2 / err.response
    true false

> Login.tsx A I
> err.response.status 401
> isAxiosError
> 4.10 4.11 response

> useNavigate() mockedNavigate
> Login.tsx

```
//          す          せ
const mockedNavigate = vi.fn();
// react-router-dom            し  え
vi.mock("react-router-dom", async () => {
  //                す
  const actual = await vi.importActual("react-router-dom");
  //          、useNavigate   け   せ
  return { ...actual, useNavigate: () => mockedNavigate };
});
```

1

- **`const mockedNavigate = vi.fn();`** Login.tsx navigate("/")
  mockedNavigate "/"

- **`vi.mock("react-router-dom", async () => { ... })`**
  MemoryRouter **Link**

- **`const actual = await vi.importActual("react-router-dom");`** `vi.importActual`
  react-router-dom MemoryRouter Link
  useNavigate actual
  - await 05 / 04

- **`return { ...actual, useNavigate: () => mockedNavigate };`** ...actual
  ** useNavigate **
  - ...actual 05
  - useNavigate: () => mockedNavigate useNavigate() mockedNavigate

> MemoryRouter Link
> useNavigate

C 1

```
// 「Login Component」 いう
describe("Login Component", () => {
  //  、 こ う
  beforeEach(() => {
    //  し
    vi.clearAllMocks();
  });
```

- `describe("Login Component", () => { ... })` `describe`
  `it` 1 C
  C
- `beforeEach(() => { ... })` `beforeEach`
- `vi.clearAllMocks();`

`mockedNavigate`

```
// ①                    OK
it("            し                  が    さ    こ  ", () => {
  // ② 【    /    】        く
  render(
    //
    <MemoryRouter>
      //
      <Login />
    </MemoryRouter>
  );

  // ③ 【    】    しがあ  か
  expect(screen.getByRole("heading", { name: "        " })).toBeInTheDocument();
  // ④ 【    】      があ  か
  expect(screen.getByLabelText("          ")).toBeInTheDocument();
});
```

1

- **`it("ログイン見出しと従業員番号入力欄が表示されること", () => {`** `it`
  1
  2 `() => { ... }`
- **`render(<MemoryRouter><Login /></MemoryRouter>)`** `render` `Login`
  `MemoryRouter` 4.1
- **`expect(screen.getByRole("heading", { name: "ログイン" })).toBeInTheDocument();`**
  1 **`heading`** " "
  **
  - `screen.getByRole(...)` `"heading"` `<h1>` `<h6>`
  - `{ name: "        " }`
  - `.toBeInTheDocument()` -
  - `Login.tsx` ・ `<h2>        </h2>` `03` 3

> H <button> button <h1>～ <h6> heading <input> textbox
> 2

- `expect(screen.getByLabelText("従業員番号")).toBeInTheDocument();` 2
  - `screen.getByLabelText(" ")` `<label> </label>`
  - `Login.tsx` `<label htmlFor="numberInput"> </label>` `<input id="numberInput">` `htmlFor` `id` `03` 3

> `03`
> 15

`/`

> A I
> `/`

```
// 
it(                                  す  こ  ", async () => {
  /                                      す
  m                                  ValueOnce({

  },

  }

  /
  r

  )

  /
  c                                  belText("          ");
  /
  f                                  rget: { value: "12345" } });

  /
  c                                  ole("button", { name: "        " });
  /
  f

  /
  a                                      ず
                                     aveBeenCalledWith("/");
  }
});
```

1

- 遷移すること", async () => { async
  await
  0
- dValueOnce({ data: { status: "success" }
  - data: { statu } } 1
  - await api.post(...)

> **Once**
> Once
> 1 post
> beforeEach 4.6

- **render(...)** 4.7
- **const input = screen.getByLabelText("従業員番号");** input
- **fireEvent.change(input, { target: { value: "12345" } });** 12345
  - fireEvent.change(...)
  - { target: { value: "12345" } } "12345" 12345 Login.tsx onChange 05
- **const button = screen.getByRole("button", { name: "ログイン" });** Login.tsx <button type="submit"> </button> 03 3
- **fireEvent.click(button);** handleSubmit api.post(...)
- **await waitFor(() => { expect(mockedNavigate).toHaveBeenCalledWith("/"); });**
  - waitFor(() => { ... }) A I waitFor
  - expect(mockedNavigate).toHaveBeenCalledWith("/") "/" toHaveBeenCalledWith( )
  - Login.tsx navigate("/")

> async / await / waitFor 04

A I status: "error"

```
it("                が   さ   こ  ", async () => {
  // ①【   】post          す  が…
  mockedApi.post.mockResolvedValueOnce({
    //          「error」
    data: { status: "error", message: "      が    か    せ  " },
  });

  render(
    <MemoryRouter>
      <Login />
    </MemoryRouter>
  );

  const input = screen.getByLabelText("          ");
  // ②     し  い
  fireEvent.change(input, { target: { value: "99999" } });

  const button = screen.getByRole("button", { name: "        " });
  // ③
  fireEvent.click(button);

  // ④【   】alert    が
  expect(await screen.findByRole("alert")).toHaveTextContent(
    //        がこ        あ  こ
    "      が    か    せ  "
  );
});
```

- `mockResolvedValueOnce({ data: { status: "error", message: "従業員が見つかりません" } })` 4.8 `mockResolvedValueOnce` ** `status` `"error"` **
  `Login.tsx` `message`
- `fireEvent.change(input, { target: { value: "99999" } });` 99999

- `expect(await screen.findByRole("alert")).toHaveTextContent("従業員が見つかりません");`
  `findByRole`
  - `screen.findByRole("alert")` `alert`
    `find` 5
  - `await`
  - `Login.tsx` `<div role="alert">{errorMessage}</div>` `03` 3
    `role="alert"` `findByRole("alert")`
  - `.toHaveTextContent("　が　か　せ　")`
    -

> **`getByRole`** **`findByRole`**
> `getBy` `findBy`
> 5

`/login`

> A I 401
> `/login`

```
it("401           /login       す   こ   ", async () => {
  // ①  【     】post  「       reject  」す
  mockedApi.post.mockRejectedValueOnce({
    //              。HTTP            401
    response: { status: 401 },
  });

  render(
    <MemoryRouter>
      <Login />
    </MemoryRouter>
  );

  const input = screen.getByLabelText("           ");
  fireEvent.change(input, { target: { value: "12345" } });

  const button = screen.getByRole("button", { name: "         " });
  fireEvent.click(button);

  // ②  【     】
  await waitFor(() => {
    //    "/login"        し     ず
    expect(mockedNavigate).toHaveBeenCalledWith("/login");
  });
});
```

- `mockedApi.post.mockRejectedValueOnce({ response: { status: 401 } });`
  `mockRejectedValueOnce`
  - `{ response: { status: 401 } }` `response` 4.4
    `isAxiosError` `true` `status: 401`
    H `07`
  - `Login.tsx` 401 `/login`
    C A DE. 401

> H 1
> `07`

- `await waitFor(() => { expect(mockedNavigate).toHaveBeenCalledWith("/login"); });`
  `"/login"` 4.8
  `/login`

401 500

A I 500

401

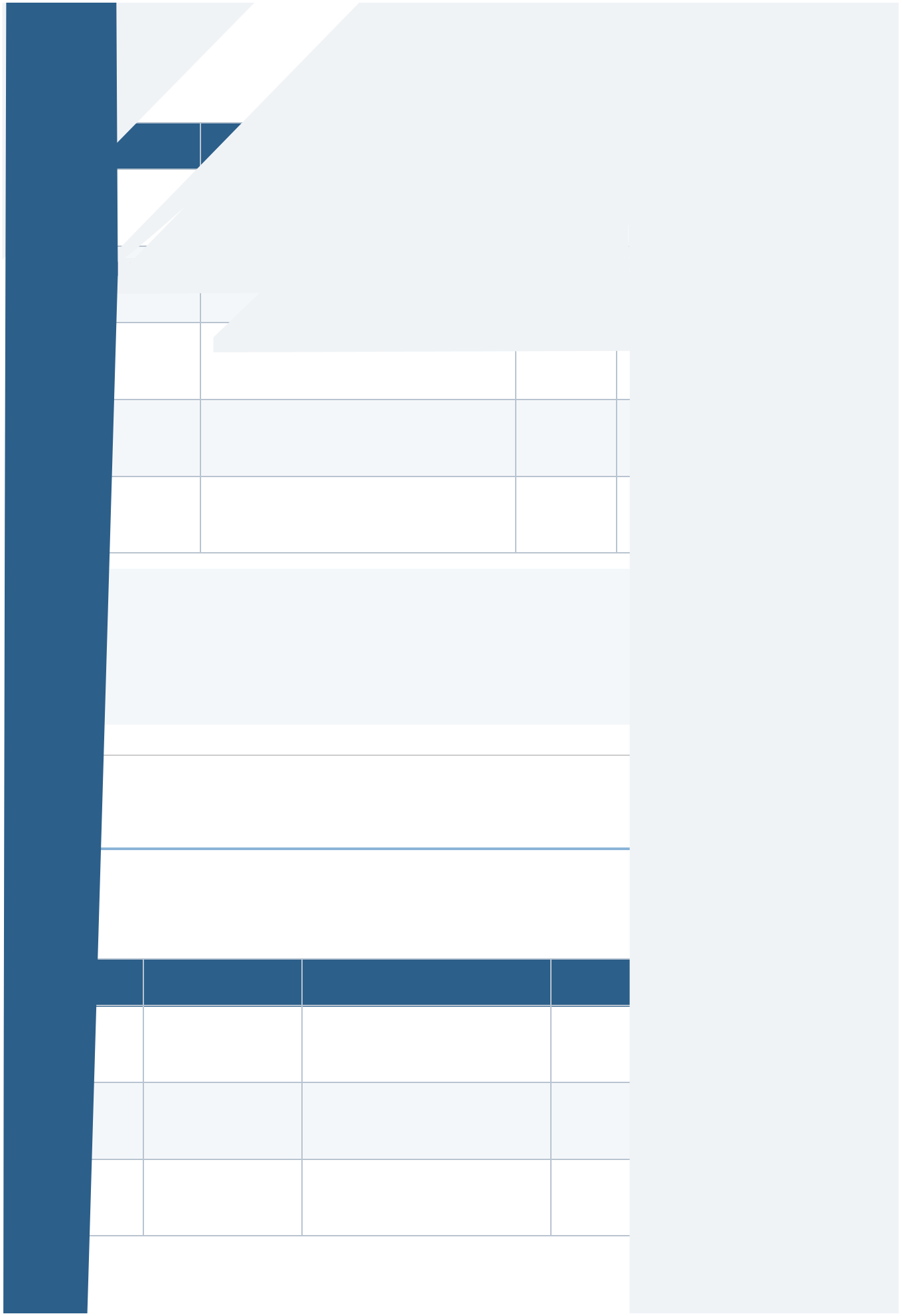
```
it("401      API                                    が    さ    こ  ", async () => {
  // ①  【    】post      す
  mockedApi.post.mockRejectedValueOnce({
    //  500
    response: { status: 500, data: { message: "              " } },
  });

  render(
    <MemoryRouter>
      <Login />
    </MemoryRouter>
  );

  const input = screen.getByLabelText("          ");
  // ②
  fireEvent.change(input, { target: { value: "12345" } });
  // ③                1
  fireEvent.click(screen.getByRole("button", { name: "        " }));

  // ④
  expect(await screen.findByRole("alert")).toHaveTextContent("              ");
});
```

- `mockRejectedValueOnce({ response: { status: 500, data: { message: "サーバーエラー" } } });` `status: 500` `data.message` `Login.tsx` 401 `message`
- `fireEvent.click(screen.getByRole("button", { name: "ログイン" }));` `getByRole(...)` `fireEvent.click(...)`
- `expect(await screen.findByRole("alert")).toHaveTextContent("サーバーエラー");` 4.9 `findByRole("alert")`



```
getBy   … 「  ここ  あ   」          。  け
findBy  … 「                」  しず          す。
queryBy … 「あ  か  いか  え  」  く        ず null     す
```

- **`screen.getByLabelText("          ")`** 4.7 , 4.8
  `getBy`
- **`screen.getByRole("button", { name: "        " })`** 4.8
  `getBy`
- **`screen.findByRole("alert")`** 4.9 , 4.11
  `getBy` `findBy`

> 3

> ⚠️ A I `getByRole("alert")`
> `findByRole`
> `findBy` `getBy`

3

>

```
// ① Vitest          く
import { defineConfig } from 'vitest/config';
// ② React    う
import react from '@vitejs/plugin-react';

// ③
export default defineConfig({
  // ④ React                    JSX        き
  plugins: [react()],
  // ⑤                ここか
  test: {
    // ⑥ describe/it/expect    import   し    え    う
    globals: true,
    // ⑦                す        §2.1
    environment: 'jsdom',
    // ⑧
    setupFiles: './src/setupTests.ts',
  },
});
```

1

-

1

```
// jest-dom adds custom jest matchers for asserting on DOM nodes.   // jest-dom  DOM
       す
// allows you to do things like:                                    //   え       う
が  き
// expect(element).toHaveTextContent(/react/i)                      //       が
    か
// learn more: https://github.com/testing-library/jest-dom         //   しく
// jest-dom
import '@testing-library/jest-dom';
```

- **`import '@testing-library/jest-dom';`** `toBeInTheDocument()` `toHaveTextContent()`
  `vite.config.ts` `setupFiles`

> `Login.test.tsx` 4 `import "@testing-library/jest-dom/vitest";`
> `setupTests.ts`

> `npm`

```
"scripts": {
  "start": "react-scripts start",   //
  "build": "react-scripts build",   //
  "test": "vitest",                 // ①        「          」                  し

  "test:ui": "vitest --ui",         // ②        UI  き
  "test:run": "vitest run",         // ③        「1   け」   し       CI
  "eject": "react-scripts eject"
},
```

- `"test": "vitest"` `npm test`

- `"test:ui": "vitest --ui"`
- `"test:run": "vitest run"`

CI 13

`devDependencies`

```
"devDependencies": {
  "@testing-library/jest-dom": "^6.9.1",      //                §6.2
  "@testing-library/react": "^16.3.1",        // RTL      render/screen/fireEvent…
  "@vitejs/plugin-react": "^5.1.2",           // React  Vitest    う          §6.1
  "@vitest/coverage-v8": "^4.1.2",            //                 け     し  か
  "jsdom": "^27.3.0",                          //               §2.1
  "vitest": "^4.0.16",                         //
  ...
}
```

`07`

`dependencies`

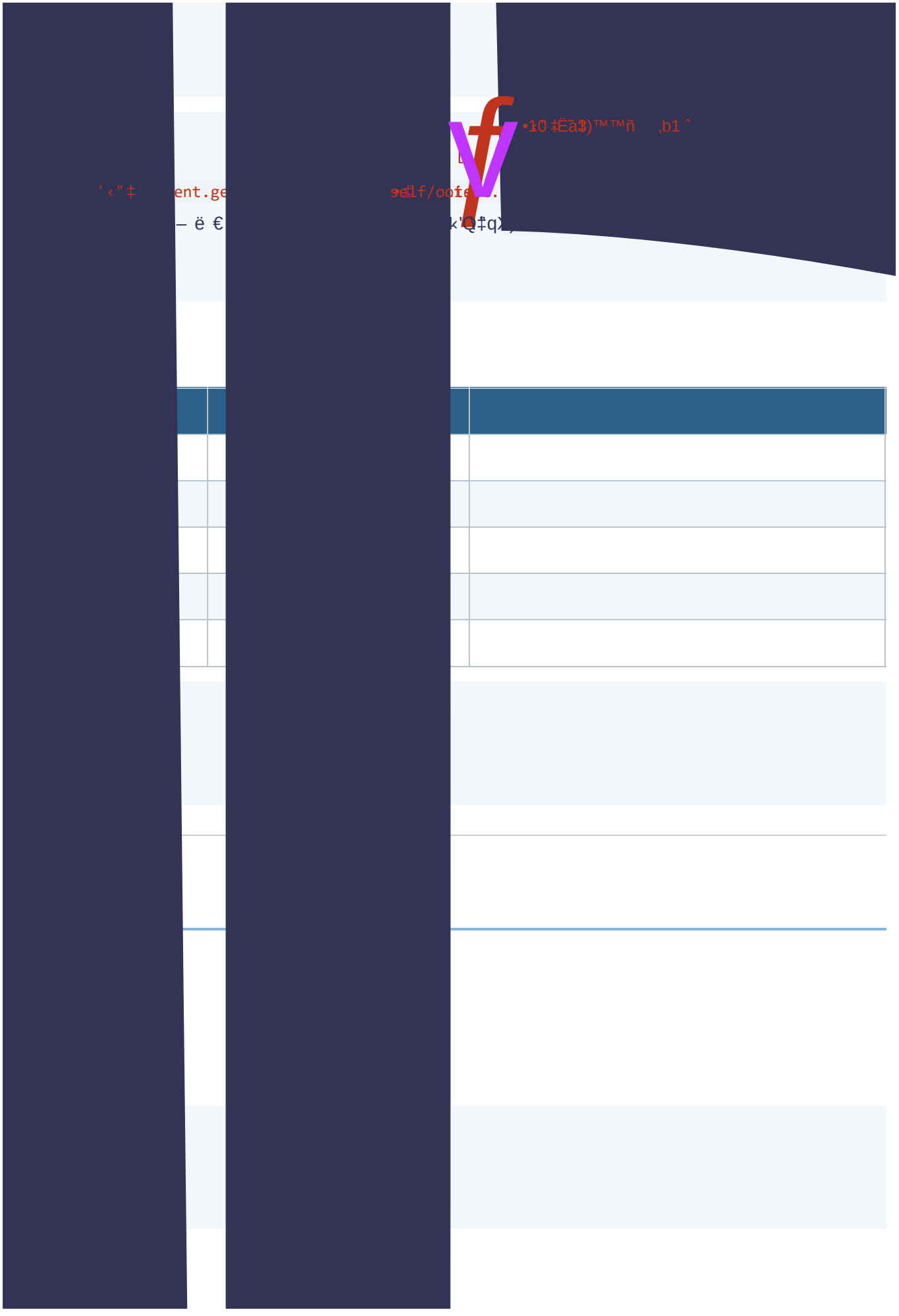
`@vitest/coverage-v8`

100%

D / 3

D

`07`



```
# ① Django
from django.test import TestCase
# ②   せ            API      し
from rest_framework.test import APIClient
# ③ HTTP                  200,401…
from rest_framework import status
# ④                    DB
from kosu.models import (
    member, Business_Time_graph, kosu_division, def_choice,
    team_member, administrator_data, inquiry_data, AsyncTask, History
)
# ⑤
from kosu.views.serializers import (
    MemberSerializer, KosuSerializer, DefSerializer, DefChoiceSerializer,
    InquirSerializer, AdministratorSerializer, TaskSerializer, HistorySerializer
)
# ⑥
from kosu.utils.main_utils import validate_employee_no_logic, get_all_model_names_in_myapp
# ⑦
from kosu.utils.kosu_utils import (
    time_index, break_time_process, kosu_write, break_time_delete,
    break_time_write, detail_list_summarize, judgement_check, kosu_sort,
    break_get, parse_break_time, get_week_of_month, kosu_division_dictionary,
    get_def_library_data, create_kosu_basic
)
# ⑧ JSON                  う
import json
# ⑨ OS
import os
# ⑩
import tempfile
# ⑪         う  dt_module  いう
import datetime as dt_module
# ⑫
from datetime import date, timedelta
# ⑬ Python
from unittest.mock import patch, MagicMock
```

- `from django.test import TestCase` 7
- `from rest_framework.test import APIClient` `self.client`
- `from rest_framework import status` `status.HTTP_200_OK`
  `200`

- `member` `Business_Time_graph` DB
  C A DE. .
- `06`
- `test_backend.py`
- `from unittest.mock import patch, MagicMock`
  3

> 1

```
def make_break_times():
    #        break_time
    """Helper to generate all required break_time fields."""
    # ①               :
    bt = {}
    # ② 1〜6          す          が6
    for i in range(1, 7):
        # ③ break_time1〜6
        bt[f'break_time{i}'] = '#10401130'
        # ④         1
        bt[f'break_time{i}_over1'] = '#15101520'
        # ⑤         2
        bt[f'break_time{i}_over2'] = '#20202110'
        # ⑥         3
        bt[f'break_time{i}_over3'] = '#01400150'
    # ⑦       が          す
    return bt
```

1

- `bt = {}` `04`
- `for i in range(1, 7):` `i` 1, 2, 3, 4, 5, 6 `range(1, 7)` 1 7
  1 6 6
  `member`
- `f'break_time{i}'` `f'...{i}...'` `i` `i=1`
  `'break_time1'` `i=2` `'break_time2'`

`'#10401130'` 10:40 11:30 `#`
`08`

- **`return bt`** `break_time1` `break_time6` `over1～3` 24

> 24
> 1 `**make_break_times()` 1 `**`
> 8.3

>

```
# ① TestCase      し
class BaseTestCase(TestCase):
    # ②
    def setUp(self):
        # ③   せ
        self.client = APIClient()

        # Create admin settings
        # ④             1
        self.admin_settings = administrator_data.objects.create(menu_row='20')

        # Create kosu_division (needed for login)
        # ⑤                 1
        self.division = kosu_division.objects.create(
            kosu_name='TestVer', kosu_title_1='    A'
        )

        # ⑥                     §8.2
        break_times = make_break_times()

        # ⑦
        self.member = member.objects.create(
            employee_no=12345, name='          ', shop='P',
            #        あ          あ
            authority=True, administrator=True, **break_times
        )

        # ⑧             し
        self.normal_member = member.objects.create(
            employee_no=99999, name='            ', shop='P',
            #          し            い
            authority=False, administrator=False, **break_times
        )
```

1

- `class BaseTestCase(TestCase):` `TestCase` `class TestModels(TestCase)` `class TestLoginLogout(BaseTestCase)` 1
- `def setUp(self):` `setUp` `beforeEach` 4.6

> C DB

>

- `self.client = APIClient()` `self.client` `self.client.get(...)` A I `self.`

- `self.admin_settings = administrator_data.objects.create(menu_row='20')` `administrator_data` `objects.create(...)` DB 1 D `06` `menu_row='20'` `self.admin_settings`

- `self.division = kosu_division.objects.create(kosu_name='TestVer', kosu_title_1='作業A')` `kosu_division` 1 `# needed for login`

- `break_times = make_break_times()` 8.2 24

- `self.member = member.objects.create(employee_no=12345, ..., authority=True, administrator=True, **break_times)`
  - `employee_no=12345`
  - `authority=True`
  - `administrator=True`
  - `**break_times` `**` `break_time1='#10401130', break_time2=..., ...` 24 8.2

> `**` `{'a':1, 'b':2}` `**` `a=1, b=2` 1

- `self.normal_member = member.objects.create(employee_no=99999, ..., authority=False, administrator=False, **break_times)` `authority=False` `administrator=False`

> 403 10 `self.member` `self.normal_member`

A I A I

```
    # ①                    す
    def login(self, employee_no=12345):
        """Helper to set session via login endpoint."""
        # ②          API  POST     く
        response = self.client.post(
            #              URL
            '/api/login/',
            # ③              JSON         し
            json.dumps({'employee_no': employee_no}),
            # ④ 「JSON       」
            content_type='application/json'
        )
        # ⑤          す
        return response

    # ⑥       (12345)          す
    def login_as_admin(self):
        return self.login(12345)

    # ⑦     (99999)          す
    def login_as_normal(self):
        return self.login(99999)
```

1

- `def login(self, employee_no=12345):` `employee_no=12345` 12345
- `response = self.client.post('/api/login/', ..., content_type='application/json')` A I `/api/login/` `07` `Login.tsx`
- `json.dumps({'employee_no': employee_no})` `{'employee_no': 12345}` `json.dumps`
- `content_type='application/json'`

- `return response`
  200

- `login_as_admin` `login_as_normal`
  `self.login(12345)` / `self.login(99999)`
  `self.login_as_admin()`

> A I C A DE.
> - 1
> `self.login_as_admin()` 1

`test_backend.py` `TestModels` 81 154

```
# TestCase BaseTestCase い
class TestModels(TestCase):
```

- `BaseTestCase` `TestCase`
  `BaseTestCase`

> 1 (1) (2)
> 2

```
# ①
def test_member_creation_and_str(self):
    # ②
    bt = make_break_times()
    # ③ 【    /    】       1
    m = member.objects.create(
        employee_no=11111, name='        ', shop='P',
        authority=False, administrator=False, **bt
    )
    # ④ 【    】         す
    self.assertEqual(str(m), '        ')
    # ⑤ 【    】           が    さ    い
    self.assertEqual(m.employee_no, 11111)
```

1

- `def test_member_creation_and_str(self):` `test_` D `test_` `it("...")`
- `bt = make_break_times()` 8.2
- `m = member.objects.create(employee_no=11111, name='山田太郎', ...)` 1 `objects.create(...)` DB 1 `m`
- `self.assertEqual(str(m), '山田太郎')` `assertEqual(a, b)` `a` `b`
  - `str(m)` `member` `__str__` `'        '`

    > D `str(...)` (1)

- `self.assertEqual(m.employee_no, 11111)` `employee_no` `11111`

B 1
ID :

```
def test_business_time_graph_creation_and_str(self):
    bt = make_break_times()
    # ①     け
    m = member.objects.create(
        employee_no=11111, name='        ', shop='P',
        authority=False, administrator=False, **bt
    )
    # ②
    kosu = Business_Time_graph.objects.create(
        #       m        2026-01-01
        employee_no3=11111, name=m, work_day2=date(2026, 1, 1),
        tyoku2='1', over_time=0
    )
    # ③     す
    expected = f'{kosu.id}__2026-01-01:11111'
    # ④【    】       が      お  か
    self.assertEqual(str(kosu), expected)
```

- `name=m` `m`
- **`Business_Time_graph.objects.create(employee_no3=11111, name=m, work_day2=date(2026,1,1), ...)`** 1 `name=m` `work_day2=date(2026, 1, 1)` `date( , , )` `tyoku2='1'` 1 `over_time=0` 0
- **`expected = f'{kosu.id}__2026-01-01:11111'`** `kosu.id` DB ID 1 ID :
- **`self.assertEqual(str(kosu), expected)`** `str(kosu)` `expected` `__str__`

1

```
def test_kosu_division_creation(self):
    #
    div = kosu_division.objects.create(kosu_name='Ver2', kosu_title_1='TaskA')
    #          'Ver2' が        か
    self.assertIn('Ver2', str(div))
```

- **`self.assertIn('Ver2', str(div))`** `assertIn(a, b)` **`a` `b` **
  `str(div)` `'Ver2'` `assertEqual`

```
def test_def_choice_creation(self):
    #
    choice = def_choice.objects.create(def_symbol='A', def_select='        ')
    #                 が
    self.assertEqual(str(choice), 'A        ')
```

- `'A'` `'        '` `'A        '` `assertEqual`

```
def test_administrator_data_creation(self):
    #
    ad = administrator_data.objects.create(menu_row='30')
    #          '    '       か
    self.assertIn('    ', str(ad))
```

- `assertIn`

> A

```python
def test_async_task_max_records_auto_deletion(self):
    """Create 1001+ records and verify oldest are deleted."""
    # Create 1002 records
    # ① 1002
    for i in range(1002):
        AsyncTask.objects.create(
            #    task_00000, task_00001, ...
            task_id=f'task_{i:05d}', status='success'
        )
    # ②                    ず
    self.assertLessEqual(AsyncTask.objects.count(), AsyncTask.MAX_RECORDS)
    # Oldest task should have been deleted
    # ③         え  い    ず
    self.assertFalse(AsyncTask.objects.filter(task_id='task_00000').exists())
```

- **`for i in range(1002):`** 1002 `f'task_{i:05d}'` `:05d` 5 `i=0` `'task_00000'` `i=1` `'task_00001'` ID

- **`self.assertLessEqual(AsyncTask.objects.count(), AsyncTask.MAX_RECORDS)`**
  `assertLessEqual(a, b)` `a ≤ b`
  `MAX_RECORDS` `AsyncTask`

- **`self.assertFalse(AsyncTask.objects.filter(task_id='task_00000').exists())`**
  `assertFalse(x)` `x`
  `...filter(task_id='task_00000').exists()`
  `False`

>

```
def test_history_save_works(self):
    #       1
    h = History.objects.create(
        #       =CREATE,       =member
        operation='CREATE', table_name='member',
        #       ID=1,          =
        record_id=1, changes={'field': 'value'}
    )
    #       が                  か
    self.assertEqual(str(h), 'CREATE on member (ID: 1)')
```

- H `'CREATE on member (ID: 1)'` `changes={'field': 'value'}` `06`

`test_backend.py`

> -
> -
>
> 401 403

`TestMainMenuViews` 3

> A I (1) 401 (2) 200 (3)
> 403 3

```
# ①  き
def test_menu_get_without_login(self):
    #  GET  し い
    response = self.client.get('/api/main_menu/')
    #  → 401が  ず
    self.assertEqual(response.status_code, 401)
```

- A I `response.status_code`

```
# ②  き
def test_menu_get_with_login(self):
    #  §8.4
    self.login_as_admin()
    #  GET
    response = self.client.get('/api/main_menu/')
    #  → 200
    self.assertEqual(response.status_code, 200)
    #  'login_data' が  か
    self.assertIn('login_data', response.data)
    #  'admin_data' が  か
    self.assertIn('admin_data', response.data)
```

- `self.login_as_admin()` GE
  `response.data` `'login_data'` `'admin_data'`
  `assertIn`

```
# ③       し       が
def test_member_menu_without_authority(self):
    #
    self.login_as_normal()
    #       が              GET
    response = self.client.get('/api/member_menu/')
    #    → 403
    self.assertEqual(response.status_code, 403)
```

- `login_as_normal`
  `authority=False`

> 1 A I
> 3
> `test_backend.py`

`TestMemberViews`

```
#              き こ
def test_member_new_post_creates_member(self):
    # ①
    self.login_as_admin()
    # ②
    data = {
        'employee_no': 55555, 'name': '       ', 'shop': 'P',
        'authority': False, 'administrator': False,
    }
    # ③    API  POST
    response = self.client.post('/api/member_new/', data, format='json')
    # ④ 201
    self.assertEqual(response.status_code, 201)
    # ⑤ DB              か
    self.assertTrue(member.objects.filter(employee_no=55555).exists())
```

- **`self.client.post('/api/member_new/', data, format='json')`** A I
  `format='json'` 8.4 `json.dumps` A IC

- `self.assertEqual(response.status_code, 201)`
  200
- `self.assertTrue(member.objects.filter(employee_no=55555).exists())` `assertTrue(x)` `x`
  55555 A I 200

```
#            き  こ
def test_member_delete(self):
    self.login_as_admin()
    # ①     API  DELETE
    response = self.client.delete(
        f'/api/member_delete/{self.normal_member.employee_no}/'
    )
    # ② 204                  し
    self.assertEqual(response.status_code, 204)
    # ③ DBか    え  か
    self.assertFalse(member.objects.filter(employee_no=99999).exists())
```

- `self.client.delete(...)` DE E E A I `07`
- `assertEqual(response.status_code, 204)`
- `assertFalse(...exists())` DB `False`

> H A I
> `07`

D /
`self.assertXXX(...)`

`assertEqual(a, b)`		`assertEqual(str(m), '  ')`
`assertNotEqual(a, b)`		

`assertTrue(x)`		`assertTrue(...exists())`
`assertFalse(x)`		`assertFalse(...exists())`
`assertIn(a, b)`		`assertIn('login_data', response.data)`
`assertNotIn(a, b)`		
`assertIsNone(x)`		`assertIsNone(err)`
`assertIsNotNone(x)`		`assertIsNotNone(err)`
`assertLessEqual(a, b)`		`assertLessEqual(count, MAX)`
`assertGreaterEqual(a, b)`		`assertGreaterEqual(week, 1)`
`assertRaises(  )`		`with self.assertRaises(ValueError):`

> **`assertRaises`**
>
> ```
> with self.assertRaises(ValueError):          # こ       ValueError が  き  こ
>     parse_break_time(None, '...', jst)        # None     す              ず
> ```
>
> `with`

`expect(a).toBe(b)`	`assertEqual(a, b)`	
`expect(x).toBeTruthy()`	`assertTrue(x)`	
`expect(x).toBeFalsy()`	`assertFalse(x)`	
`expect(el).toBeInTheDocument()`	D	
`expect(el).toHaveTextContent(s)`	`assertIn(s, ...)`	
`expect(fn).toHaveBeenCalledWith(x)`		

> `expect(...).toBe(...)` `self.assertEqual(...)`

`hozen_another/test_settings.py` 31

> (1) DB
> (2) (3)

```
# ①          す      き  ぐ
from .settings import *

# ② DB           き
DATABASES = {
    'default': {
        #    SQLite    う  PostgreSQL    く
        'ENGINE': 'django.db.backends.sqlite3',
        #             DB             さ  い
        'NAME': ':memory:',
    }
}

# Disable signals during tests to avoid side effects
# ③ 「           」                 え
TESTING = True

# Disable django-q cluster requirement for tests
# ④
Q_CLUSTER = {
    'name': 'test',
    #    'sync': True               「           」
    'sync': True,
    'timeout': 60,
    'orm': 'default',
}

# Custom test runner that creates tables from models directly (bypasses migrations)
# ⑤                        す      け
class DisableMigrations:
    """Fake migration module that tells Django to skip migrations and use syncdb."""
    def __contains__(self, item):
        return True
    def __getitem__(self, item):
        return None

# ⑥                       し、      か
MIGRATION_MODULES = DisableMigrations()
```

1

- `from .settings import *` `settings.py`
- `sqlite3` `:memory:` C A DE.

> DB DB

- `TESTING = True`
  D
  C A DE. `signals.py`

- `Q_CLUSTER` `'sync': True` D -
  C A DE. `'sync': True`

> `sync: True`

- `DisableMigrations` DB `06`
  `makemigrations` / `migrate`

> `max_length`

> `--`
> `settings=hozen_another.test_settings` 13

`frontend/`

```
#         1    し                    こ
npm run test:run
#                                  。        こ
npm test
#         UI
npm run test:ui
```

> **`npm run test:run`**
>
> ```
> ✓ src/tests/MainPage/Login.test.tsx (5 tests) 123ms
>   ✓ Login Component >            し                が    さ    こ
>   ✓ Login Component >                /       す  こ
>   ✓ Login Component >                              が    さ    こ
>   ✓ Login Component > 401          /login       す  こ
>   ✓ Login Component > 401      API                                  が    さ    こ
>
> Test Files  1 passed (1)
>      Tests  5 passed (5)
> ```
>
> `✓` `passed` `describe`
> C `it`

`manage.py`

```
python manage.py test --settings=hozen_another.test_settings
```

- `python manage.py test` D
- `--settings=hozen_another.test_settings`
  DB

```
# TestModels         け   す
python manage.py test kosu.tests.test_backend.TestModels --
settings=hozen_another.test_settings

# 1                    け   す
python manage.py test kosu.tests.test_backend.TestModels.test_member_creation_and_str --
settings=hozen_another.test_settings
```

▼ こう表示されれば成功:

---

120 4.231

D ' '...

```
- `.`          1  が **      1        **。
-        `OK` が「**        **」       。
- `Creating test database` / `Destroying test database` … §12        お  、        DB
        し  い  す。
```

⚠ `OK` `FAILED (failures=1)` `.` `F` `E` 14

```
======================================================================
FAIL: test_member_creation_and_str (kosu.tests.test_backend.TestModels)
----------------------------------------------------------------------
Traceback (most recent call last):
  File ".../test_backend.py", line 88, in test_member_creation_and_str
    self.assertEqual(str(m), '        ')
AssertionError: 'member object (1)' != '        '
----------------------------------------------------------------------
```

1. **`FAIL: test_member_creation_and_str`**
2. **`line 88, in ...`**
3. **`self.assertEqual(str(m), '山田太郎')`**
4. **`AssertionError: 'member object (1)' != '山田太郎'`** `'member object (1)'` `'        '` `!=`

`member` `__str__`

```
 FAIL  src/tests/MainPage/Login.test.tsx > Login Component >                  /       す  こ
AssertionError: expected "mockedNavigate" to be called with arguments: [ '/' ]

Number of calls: 0
```

- `>`
- **`expected ... to be called with [ '/' ]`** `/`
- **`Number of calls: 0`**

`waitFor`

```
1.                    …
2. 「     が  しい」か    …                    す  き     あ
   「    が      」か
3a.     が              →            す          しい
3b.     が              →                  せ     す
4.       し              … 1   すご     し  す
```

> ⚠️

> `/`
>
> `/home` `/`
>
> `/home`

`Login.test.tsx` 2 4.8 ＡＩ

>

```
// ①   しい
it("          が              API        いこ  ", () => {
  // ②         く
  render(
    <MemoryRouter>
      <Login />
    </MemoryRouter>
  );
  // ③
  fireEvent.click(screen.getByRole("button", { name: "        " }));
  // ④       post          い  い  ず
  expect(mockedApi.post).not.toHaveBeenCalled();
});
```

- `render` `fireEvent` `expect`

- `expect(...).not.toHaveBeenCalled()` `not`

> `Login.tsx` `required`

`test_backend.py` 10.2 A I 401

>

```
1.                    け          …   じ          /    /    …      す
2.          し          え          … it("...") / def test_xxx
3.                        せ        … mock... / data = {...}
4.                せ                …     す          くURL
5.                せ                … expect / assert
6.    し          か                … npm run test:run / python manage.py test ...
```

> `test_backend.py`
>
> 1

`python manage.py test` DB	`--settings=hozen_another.test_settings`	13.2
`Cannot find ... TitleRogo.png`		`vi.mock("...png", () => ({ default: "..." }))` 4.2
`useNavigate() may be used only in the context of a Router`		`<MemoryRouter>` `<Login />` 4.1
	`getByRole("alert")`	`findByRole("alert")` 5
`toHaveBeenCalledWith` `Number of calls: 0`	`waitFor`	`await waitFor(...)` `mockResolvedValueOnce` 4.8
		`beforeEach(vi.clearAllMocks)` / D `setUp` DB 4.6, 8.3

`toBeInTheDocument`	-	`setupTests.ts` `setupFiles` 6.2
403 200		`login_as_normal()` 10
201/204 400		`data` 10.3
`AssertionError: ... != ...`		14

---

1. `frontend/` `npm run test:run` 5
2. `python manage.py test kosu.tests.test_backend.TestModels --settings=hozen_another.test_settings` `OK`
3. `Login.test.tsx` 2 `toHaveBeenCalledWith("/")` `toHaveBeenCalledWith("/home")`
4. **`getBy`** **`findBy`** 3 `findByRole("alert")` `getByRole("alert")`
5. `test_backend.py` `TestMemberViews` 200 401 403 3 `login`
6. 15.2 `TestModels` `def_choice` `def_symbol` `assertEqual` 1

-
- A A A

- Login.test.tsx 132 1 vi.mock A I describe / it / beforeEach render / screen / fireEvent / waitFor getByRole / getByLabelText / findByRole mockResolvedValueOnce / mockRejectedValueOnce
- **getBy** **findBy** **queryBy**
- **D TestCase APIClient** test_backend.py make_break_times BaseTestCase login TestModels 1 assertEqual / assertIn / assertTrue / assertFalse / assertLessEqual
- A I 401 403 201 204
- test_settings.py **npm run test:run** **python manage.py test --settings=hozen_another.test_settings** ✓ / OK
-
- 1

EAD E.